

## Diagrama de la arquitectura de microservicios

La solución propuesta se estructura bajo un patrón de arquitectura de microservicios desacoplados para resolver de forma eficiente y escalable las necesidades del cliente. La arquitectura consta de los siguientes componentes integrados a través de una API REST.

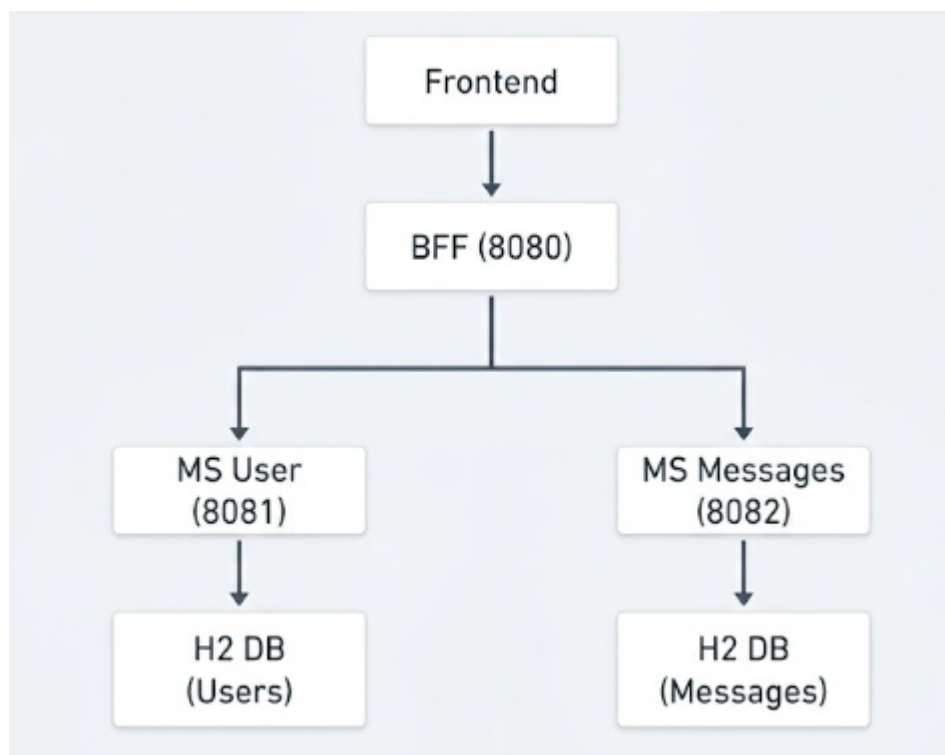
**Componente Frontend:** Interfaz de usuario construida sobre un framework moderno que interactúa de manera exclusiva con el punto de entrada del backend.

**Backend For Frontend (BFF - Puerto 8080):** Actúa como una puerta de enlace de API (*API Gateway* adaptada). Centraliza las peticiones del cliente, realiza la orquestación de llamadas y unifica las respuestas provenientes de los microservicios internos, optimizando la comunicación y la seguridad.

**Microservicio de Usuarios (MS User - Puerto 8081):** Encargado de toda la lógica de negocio, gestión y control del dominio de cuentas y perfiles de usuario.

**Microservicio de Mensajes (MS Messages - Puerto 8082):** Responsable de administrar la persistencia, envío y flujo de las comunicaciones e historiales de mensajes en el sistema

Y para cumplir con el requisito de persistencia se implementó un motor de base de datos H2 el cual genera una base de datos tanto para usuarios como para mensajes



**Desarrollo e Integración de Componentes Frontend:** Para la interfaz de usuario se diseñó y desarrolló una aplicación utilizando la biblioteca **React**, cumpliendo con el objetivo de proveer una experiencia interactiva, modular y eficiente en tiempo real



## Persistencia de datos:

Para la capa de datos del proyecto hemos seleccionado una solución basada en Spring Data JPA como framework de persistencia y H2 como motor de base de datos en memoria. Esta combinación permite un desarrollo ágil, desacoplado y con una configuración simple y accesible

## Herramientas utilizadas:

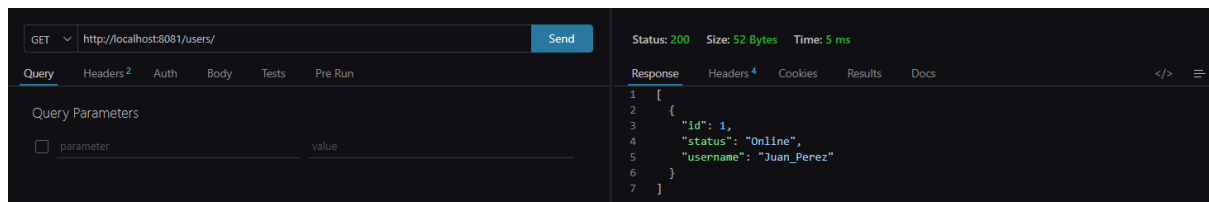
**JUnit:** Es el framework estándar de pruebas para Java. Se utiliza para estructurar, ejecutar y validar las pruebas unitarias del código de manera automatizada mediante aserciones (ej. verificar que un método retorne el resultado esperado).

**Spring Boot Test:** Es un módulo que provee anotaciones y utilidades específicas para integrar el contexto de Spring en las pruebas. Permite realizar pruebas de integración, levantar servidores web de prueba y simular el comportamiento de controladores REST y repositorios.

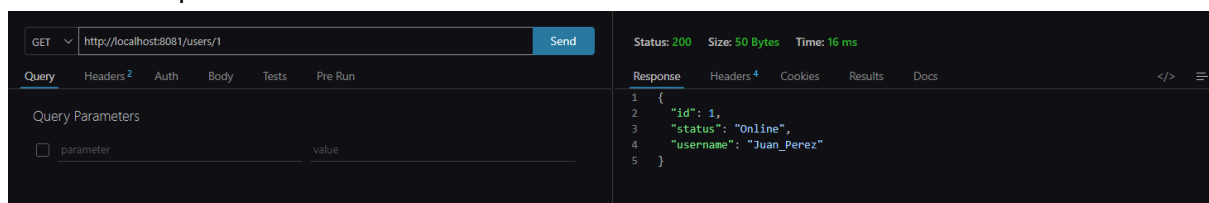
**JaCoCo (Java Code Coverage):** Es una herramienta que analiza el código durante la ejecución de las pruebas para medir el porcentaje de cobertura. Genera informes visuales que indican exactamente qué líneas de código, ramas y clases fueron probadas y cuáles faltaron por ejecutar.

## Pruebas Postman:

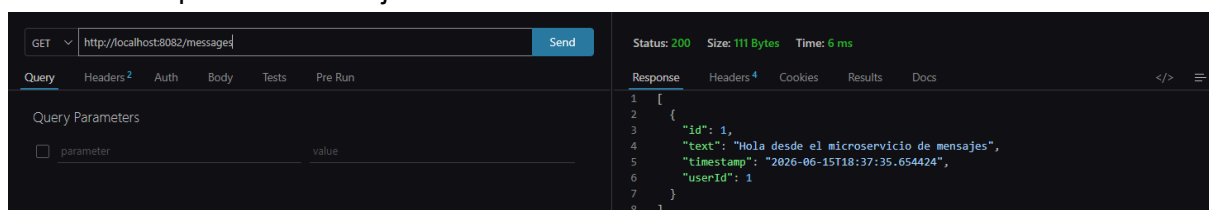
Método GET para ver usuarios:



Método GET para buscar usuario mediante ID:



Método GET para ver mensajes:



Método POST para crear mensajes:

POSThttp://localhost:8082/messagesSend

QueryHeadersAuthBody1TestsPre Run

QueryParametersJSONXMLTextFormForm-encodeGraphQLBinary

JSON ContentFormat

```
1 {
2   "text": "Mensaje de prueba método POST",
3   "timestamp": "2026-06-15T18:37:35.654424",
4   "userId": 1
5 }
```

Status: 200Size: 101 BytesTime: 41 ms

ResponseHeaders4CookiesResultsDocs

```
1 {
2   "id": 2,
3   "text": "Mensaje de prueba método POST",
4   "timestamp": "2026-06-15T18:57:25.7479635",
5   "userId": 1
6 }
```

Swagger

Usuarios:

localhost:8081/swagger-ui/index.html#

Swagger

/v3/api-docsExplore

OpenAPI definitionv0OAS 3.1

Servers

http://localhost:8081 - Generated server url

user-controller

GET/users

GET/users/{id}

Schemas

User > Expand all object

GET/users

Parameters

No parameters

ExecuteClear

Responses

Curl

curl -X 'GET' \
'http://localhost:8081/users' \
-H 'accept: \*/\*'

Request URL

http://localhost:8081/users

Server response

CodeDetails

200

Response body

{
 "id": 1,
 "status": "Online",
 "username": "Juan\_Perez"
}

Response headers

connection: keep-alive
content-length: 52
content-type: application/json
date: Mon, 15 Jun 2026 23:04:15 GMT
keep-alive: timeout=60

GET /users/{id}

Parameters

Name

Description

id \* required

integer(\$int64)

1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/users/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/users/1
```

Server response

Code

Details

200

Response body

```
{
  "id": 1,
  "Status": "Online",
  "username": "Juan_Perez"
}
```

Response headers

```
connection: keep-alive
content-length: 50
content-type: application/json
date: Mon, 15 Jun 2026 23:04:42 GMT
keep-alive: timeout=60
```

## Mensajes:

localhost:8082/swagger-ui/index.html

Swagger

/v3/api-docs

Explore

OpenAPI definition

v0 OAS 3.1

v3/api-docs

Servers

http://localhost:8082 - Generated server url

message-controller

GET /messages

POST /messages

Schemas

Message > Expand all object

GET

/messages

^

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'GET' \  
'http://localhost:8082/messages' \  
-H 'accept: \*/\*'

Request URL

http://localhost:8082/messages

Server response

Code

Details

200

Response body

[  
 {  
 "id": 1,  
 "text": "Hola desde el microservicio de mensajes",  
 "timestamp": "2026-06-15T18:59:54.88847",  
 "userId": 1  
 }  
]

Download

Response headers

connection: keep-alive  
content-length: 110  
content-type: application/json  
date: Mon, 15 Jun 2026 23:06:02 GMT  
keep-alive: timeout=60

POST

/messages

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json

{  
 "userId": 1,  
 "text": "Persistencia de datos",  
 "timestamp": "2026-06-15T23:07:19.4622"  
}

Execute

Clear

Responses

Curl

curl -X 'POST' \  
'http://localhost:8082/messages' \  
-H 'accept: \*/\*' \  
-H 'Content-Type: application/json' \  
-d '{  
 "userId": 1,  
 "text": "Persistencia de datos",  
 "timestamp": "2026-06-15T23:07:19.4622"  
}'

Request URL

http://localhost:8082/messages

Backend-for-frontend:

localhost:8080/swagger-ui/index.html#/v3/api-docs

Swagger

v3/api-docs

Explore

OpenAPI definition

v0 OAS 3.1

v3/api-docs

Servers

http://localhost:8080 - Generated server url

bff-controller

POST /api/v1/chat/send

GET /api/v1/chat/dashboard

Schemas

DashboardDTO > Expand all object

POST /api/v1/chat/send

Parameters

No parameters

Request body required

application/json

```
{  "userId": 1,  "text": "Mensaje enviado desde el BFF"}
```

Execute

Clear

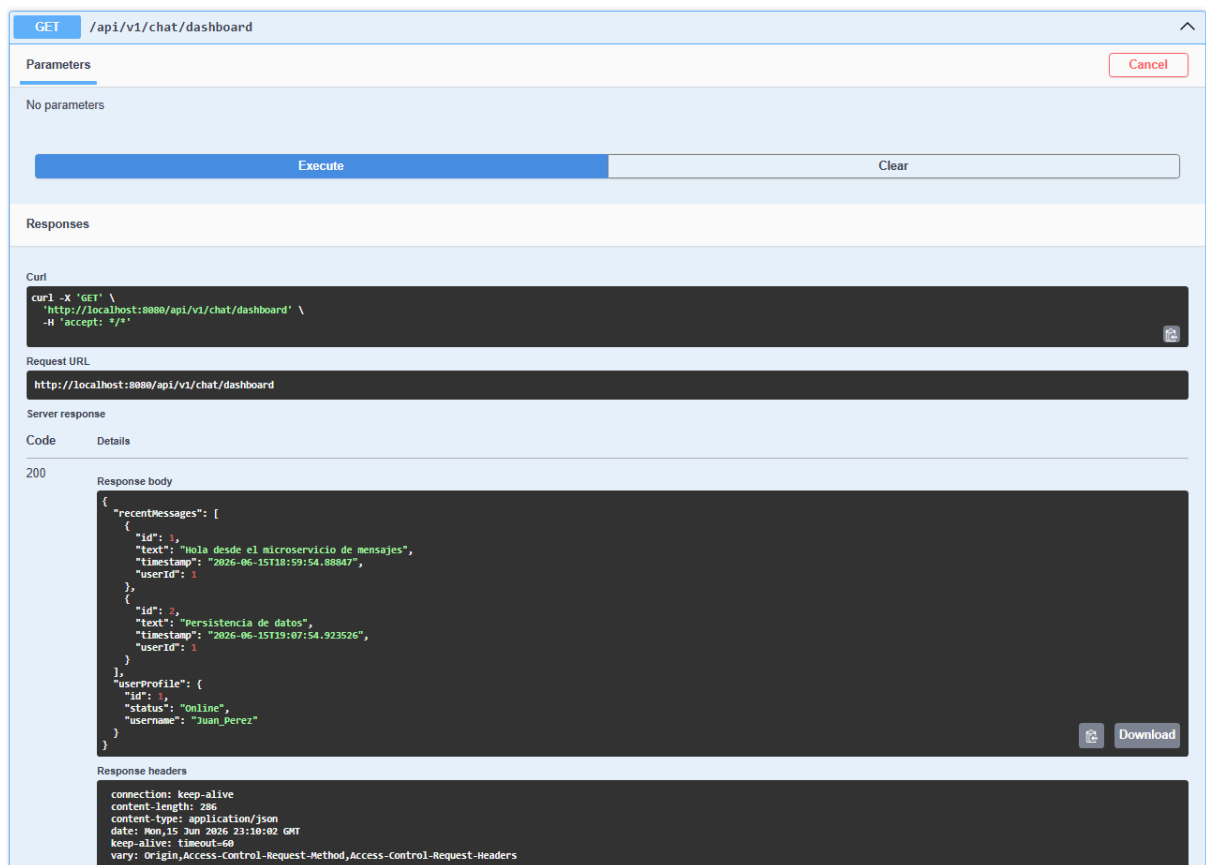
Responses

curl

```
curl -X 'POST' \  'http://localhost:8080/api/v1/chat/send' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "userId": 1,  "text": "Mensaje enviado desde el BFF"  }'
```

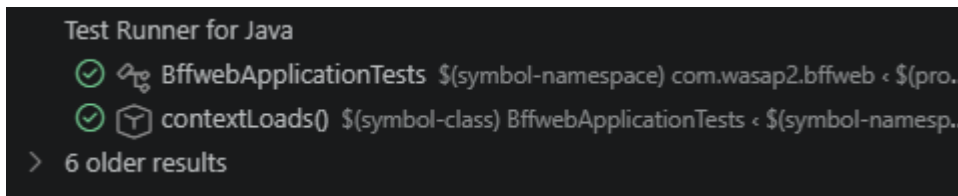
Request URL

http://localhost:8080/api/v1/chat/send

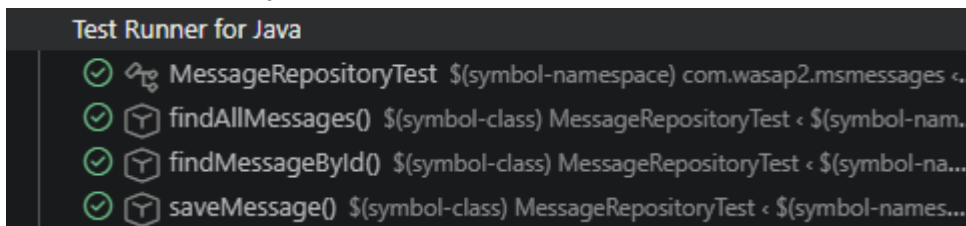


## TEST

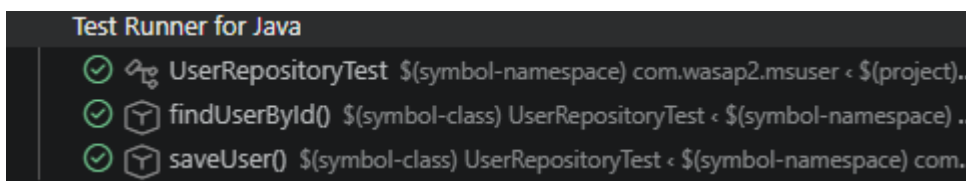
Backend-for-frontend:



Microservicio Mensajes:



Microservicio Usuarios:





**Jacoco(Análisis de cobertura):** Para generar los archivos de análisis de cobertura se debe ingresar a la carpeta del microservicio y ejecutar el siguiente comando en la terminal (./mvnw clean test), esto generará un archivo en la ruta target/site/jacoco llamado index.html donde se encontrará el análisis generado.

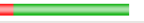
Microservicio Mensajes (70% de cobertura):

| Element                         | Missed Instructions                                                               | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods | Missed | Classes |
|---------------------------------|-----------------------------------------------------------------------------------|------|-----------------|------|--------|------|--------|-------|--------|---------|--------|---------|
| com.wasap2.msessages.controller |  | 66%  |                 | n/a  | 1      | 4    | 2      | 6     | 1      | 4       | 0      | 1       |
| com.wasap2.msessages            |  | 37%  |                 | n/a  | 1      | 2    | 2      | 3     | 1      | 2       | 0      | 1       |
| com.wasap2.msessages.model      |  | 100% |                 | n/a  | 0      | 1    | 0      | 5     | 0      | 1       | 0      | 1       |
| Total                           | 14 of 47                                                                          | 70%  | 0 of 0          | n/a  | 2      | 7    | 4      | 14    | 2      | 7       | 0      | 3       |

Microservicio Usuarios (78% de cobertura):

| Element                      | Missed Instructions                                                               | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods | Missed | Classes |
|------------------------------|-----------------------------------------------------------------------------------|------|-----------------|------|--------|------|--------|-------|--------|---------|--------|---------|
| com.wasap2.msuser            |  | 37%  |                 | n/a  | 1      | 2    | 2      | 3     | 1      | 2       | 0      | 1       |
| com.wasap2.msuser.controller |  | 84%  |                 | n/a  | 1      | 4    | 1      | 5     | 1      | 4       | 0      | 1       |
| com.wasap2.msuser.model      |  | 100% |                 | n/a  | 0      | 1    | 0      | 4     | 0      | 1       | 0      | 1       |
| Total                        | 9 of 42                                                                           | 78%  | 0 of 0          | n/a  | 2      | 7    | 3      | 12    | 2      | 7       | 0      | 3       |

Backend-for-frontend (80% de cobertura):

| Element                      | Missed Instructions                                                               | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods | Missed | Classes |
|------------------------------|-----------------------------------------------------------------------------------|------|-----------------|------|--------|------|--------|-------|--------|---------|--------|---------|
| com.wasap2.bffweb.controller |  | 83%  |                 | n/a  | 0      | 4    | 6      | 18    | 0      | 4       | 0      | 2       |
| com.wasap2.bffweb            |  | 37%  |                 | n/a  | 1      | 2    | 2      | 3     | 1      | 2       | 0      | 1       |
| com.wasap2.bffweb.config     |  | 100% |                 | n/a  | 0      | 2    | 0      | 2     | 0      | 2       | 0      | 1       |
| Total                        | 16 of 81                                                                          | 80%  | 0 of 0          | n/a  | 1      | 8    | 8      | 23    | 1      | 8       | 0      | 4       |

## Repositorios del Proyecto:

Para mantener una mejor organización y facilitar el mantenimiento cada componente fue almacenado en un repositorio de GitHub distinto.

### Repositorio Principal:

<https://github.com/Bonjo06/wasap3-main.git>

Contiene la documentación principal del proyecto, diagrama de arquitectura, evidencia y el archivo con los enlaces a los repositorios asociados.

### Repositorio Frontend:

<https://github.com/Bonjo06/wasap3-frontend.git>

Contiene la interfaz de usuario desarrollada, se encarga de interactuar con el Backend-for-Frontend.

### Repositorio Backend-for-Frontend:

<https://github.com/Bonjo06/wasap3-backend-for-frontend.git>

Componente encargado de centralizar las peticiones provenientes del Frontend y coordinar la comunicación entre los microservicios internos.

**Repositorio Microservicio de Usuarios:**

<https://github.com/Bonjo06/wasap3-msuser.git>

Microservicio encargado de la gestión de usuarios, incluyendo persistencia de datos mediante spring data JPA y H2.

**Repositorio Microservicio de Mensajes:**

<https://github.com/Bonjo06/wasap3-msmessages.git>

Microservicio encargado de la gestión y persistencia de mensajes, implementado con spring boot, spring data JPA y H2.